

SYSTEM ARCHITECTURE SPECIFICATION

AeroBook Flight Platform

Complete Microservices Architecture & Engineering Guide

In-depth architectural breakdown of Reactive Gateway Routing, Database-per-Service Isolation, OpenFeign Inter-Service Protocols, and RabbitMQ Event Messaging.

ENTERPRISE ARCHITECTURAL PARAMETERS

API Gateway: Spring Cloud Gateway (8083)	Service Discovery: Netflix Eureka Registry (8761)	Message Broker: RabbitMQ AMQP Broker (5672)
Reactive Engine: Netty Non-Blocking WebFlux	Inter-Service REST: Spring Cloud OpenFeign	Data Tier: 6 Isolated MySQL 8 Schemas

Executive Overview

The AeroBook platform is an enterprise-class flight reservation, fleet scheduling, dynamic yield pricing, and airport operations system. To guarantee horizontal scalability, fault isolation, and independent deployability, AeroBook is designed according to modern microservices principles:

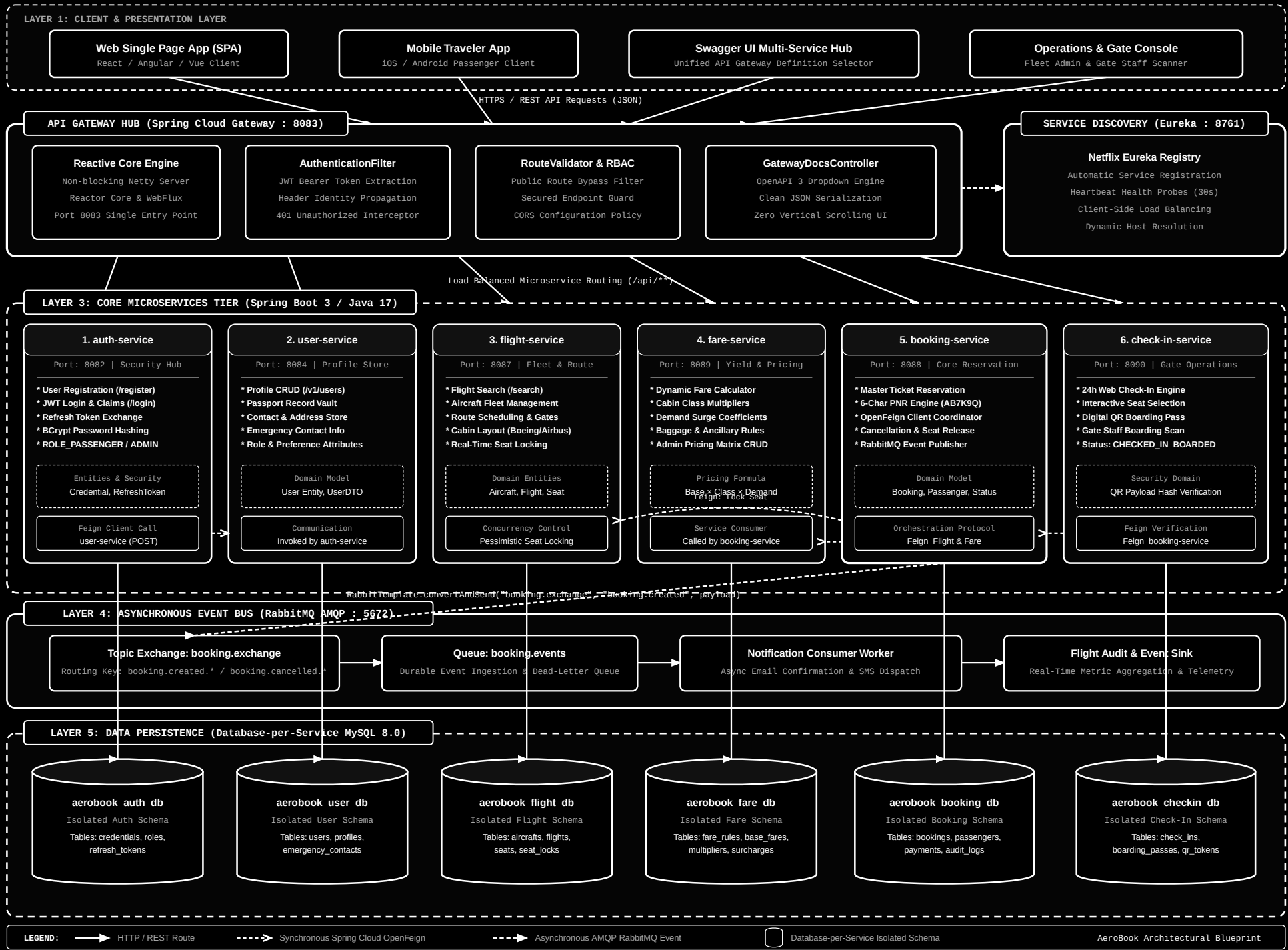
- Edge Gateway & Centralized Security: All client interaction is funneled through a reactive Spring Cloud Gateway (port 8083) that acts as the single public entry point, validating JWT Bearer tokens and enforcing Role-Based Access Control (RBAC).
- Database-per-Service Architecture: Each microservice strictly owns its dedicated MySQL schema (ports 3306), eliminating database coupling and preventing cross-domain schema dependencies.
- Dual Communication Paradigms: Synchronous low-latency queries utilize declarative Spring Cloud OpenFeign clients, while mission-critical events (booking confirmations, seat locks, notifications) publish asynchronously through RabbitMQ.

DOCUMENT TABLE OF CONTENTS

Page 2	High-Resolution Architecture Diagram (Landscape Vector) Complete 5-tier architecture blueprint in black canvas & white marker theme
Page 3	Section 1: Architecture Fundamentals & Edge Gateway Layer Netty reactive engine, JWT authentication filter, route whitelist, and Eureka
Page 4	Section 2: Core Microservices Domain Layer (Part 1) auth-service (8082), user-service (8084), flight-service (8087) & seat locking
Page 5	Section 3: Core Microservices Domain Layer (Part 2) fare-service (8089), booking-service (8088), check-in-service (8090) & QR passes
Page 6	Section 4: Inter-Service Communication & Event Messaging Spring Cloud OpenFeign RPC, RabbitMQ topic exchanges, and DLQ architecture
Page 7	Section 5: Data Persistence & Cross-Cutting Architecture Database-per-Service isolation, non-null JSON serialization, and global lints
Page 8	Section 6: Microservice Topology Matrix & Defense FAQ Complete port/DB/protocol mapping table and technical review defense answers

AEROBOOK PLATFORM - SYSTEM ARCHITECTURE DIAGRAM

Reactive Microservices Architecture • Database-per-Service • Asynchronous Event Messaging • Black & White Theme



1. Microservices Architecture Fundamentals

Core Principles of the Distributed System

In a traditional monolithic architecture, all functional modules (authentication, flight scheduling, ticket reservation, payments) reside in a single codebase sharing a single database. This creates severe deployment bottlenecks, single-point-of-failure vulnerabilities, and scaling inefficiencies.

AeroBook resolves these limitations by adopting the Microservices Architecture Pattern. The system is decomposed into small, autonomous, and loosely coupled services. Each service encapsulates a single business capability, exposes a strictly versioned REST API, maintains its own database schema, and can be scaled, deployed, or restarted without impacting neighboring services.

2. Edge Gateway & Service Discovery Tier

Spring Cloud Gateway (Port 8083) & Netflix Eureka (Port 8761)

Centralized API Gateway Hub (Port 8083)	Spring Cloud Gateway / Project Reactor & Netty
Acts as the single front door for all client traffic (Web SPA, Mobile App, Swagger UI, Admin consoles). Because downstream services reside on private networks or container subnets, the gateway shields internal topology and handles reverse-proxy routing.	
Reactive Non-Blocking Engine	Netty EventLoop & WebFlux
Unlike traditional Spring MVC servlet engines that allocate a heavy operating system thread per request, the Netty engine uses non-blocking I/O event loops. This enables the gateway to effortlessly handle thousands of concurrent requests with minimal CPU and RAM usage.	
AuthenticationFilter (JWT Security Guard)	Spring Cloud Gateway Pre-Filter
Intercepts incoming requests, extracts the "Authorization: Bearer <token>" header, decodes the JWT signature using the HMAC256 secret, validates expiration, and extracts user identity (id, email, roles). It then populates downstream headers (X-User-Id, X-User-Role) for microservices.	
RouteValidator (Public vs. Secured Whitelist)	Predicate Routing Whitelist
Maintains an explicit list of public bypass routes (/api/auth/**, /api/gateway/**, /swagger-ui/**, **/v3/api-docs/**) that bypass JWT checks. All other routes (e.g. /api/bookings, /api/admin/**) strictly enforce active credentials, rejecting unauthorized traffic with HTTP 401.	
Netflix Eureka Service Registry (Port 8761)	Dynamic Service Discovery & Client-Side Load Balancing
Provides automated discovery. When any microservice boots up, it registers its hostname, port, and IP with Eureka. Microservices emit heartbeat probes every 30 seconds. The API Gateway queries Eureka to dynamically resolve service names (e.g., lb://booking-service) without static IP hardcoding.	
GatewayDocsController & Dropdown Engine	Custom Swagger UI Multi-Service Definition Provider
Serves isolated OpenAPI 3 definitions for each microservice. Implements clean JSON serialization via io.swagger.v3.core.util.Json.mapper() with strict non-null inclusion, completely resolving Swagger UI \$ref resolver bugs and eliminating vertical scrolling.	

3. Core Microservices Domain Layer (Part 1)

Identity, Authentication, Customer Profile & Flight Inventory

1. auth-service | Security Authority & Credential Vault

Port: 8082 | Database: aerobook_auth_db | Tables: credentials, roles, refresh_tokens

- * Architectural Responsibility: Acts as the central identity provider (IdP) for the platform. Manages traveler registration, BCrypt password hashing (12 salt rounds), credential verification, JWT token issuance, and refresh token rotation.
- * Security Mechanics: Issues stateless HMAC-SHA256 signed JWT tokens containing sub (subject/userId), role (ROLE_PASSENGER, ROLE_ADMIN, ROLE_STAFF), and exp (24h expiration timestamp). Enforces unique email constraints.
- * Synchronous Integration: When a new passenger registers via POST /api/auth/register, auth-service creates the security credential record and immediately calls user-service over OpenFeign (POST /api/v1/users) to initialize the traveler profile.
- * Key Endpoints: POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh-token, GET /api/auth/credentials.

2. user-service | Traveler Profile & Demographics Vault

Port: 8084 | Database: aerobook_user_db | Tables: users, addresses, emergency_contacts

- * Architectural Responsibility: Decoupled customer information store. Completely separates personal traveler demographics (full name, phone, residential address, nationality, passport details) from login credentials.
- * Compliance & Isolation: Isolating personal data ensures that GDPR and privacy compliance standards can be enforced on user-service without altering authentication mechanisms.
- * Data Model: User entity containing primary key id, linked to Credential in auth-service via userId reference (logical foreign key, no cross-database constraint).
- * Key Endpoints: POST /api/v1/users (Profile creation), GET /api/v1/users/{id} (Profile retrieval), PUT /api/v1/users/{id} (Profile update), DELETE /api/v1/users/{id}.

3. flight-service | Aircraft Fleet & Route Scheduling

Port: 8087 | Database: aerobook_flight_db | Tables: aircrafts, flights, seats, seat_locks

- * Architectural Responsibility: Manages the physical and scheduled airline assets: aircraft models (Boeing 737, Airbus A320), seating maps, daily flight schedules, departure/arrival airports, gate assignments, and seat availability.
- * Atomic Seat Locking Algorithm: Solves the classic airline concurrency challenge (race conditions where two passengers attempt to book the last available seat simultaneously). When booking-service initiates a reservation, flight-service places a temporary 10-minute hold on the requested seat (seat_locks table). If payment fails or times out, the lock is automatically released.
- * Cabin Class Architecture: Models 4 distinct cabin categories per flight: Economy, Premium Economy, Business Class, and First Class with independent seat capacities and real-time remaining seat counters.
- * Key Endpoints: GET /api/flights/search, GET /api/flights/{flightNumber}, POST /api/flights/{flightNumber}/seats/lock, POST/GET /api/admin/aircrafts, POST/GET /api/admin/flights.

CONCURRENCY PRINCIPLE: Why Atomic Seat Locking Matters

In distributed systems, eventual consistency is acceptable for analytics, but seat inventory demands immediate consistency. By isolating seat locking inside flight-service with database row-level locking (SELECT ... FOR UPDATE) and a 10-minute TTL, AeroBook mathematically prevents double-booking across millions of concurrent searchers.

4. Core Microservices Domain Layer (Part 2)

Dynamic Fare Engine, Booking Orchestration & Airport Check-In

4. fare-service | Yield Management & Dynamic Pricing Engine

Port: 8089 | Database: aerobook_fare_db | Tables: fare_rules, base_fares, class_multipliers

- * Architectural Responsibility: Provides algorithmic fare calculation and revenue management. Rather than hardcoding fixed ticket prices, it computes quotes dynamically using real-time demand factors.
- * Dynamic Pricing Formula: Calculated as $\text{Price} = (\text{Base Route Fare} * \text{Cabin Class Multiplier} * \text{Surge Multiplier}) + \text{Baggage Surcharge}$.
- * Multiplier Specifications: Economy Class = 1.0x, Premium Economy = 1.3x, Business Class = 1.8x, First Class = 2.5x. Surge factors activate when seat availability drops below 20% or departure is within 72 hours.
- * Key Endpoints: POST /api/fares/calculate (Quote calculation), GET /api/fares/{id}, POST /api/fares (Admin rule creation), PUT /api/fares/{id}.

5. booking-service | Master Transaction & Reservation Orchestrator

Port: 8088 | Database: aerobook_booking_db | Tables: bookings, passengers, payment_logs

- * Architectural Responsibility: The central transactional engine of AeroBook. Coordinates the end-to-end ticket purchase saga across flight-service, fare-service, and external payment processors.
- * Collision-Resistant PNR Engine: Implements PnrGenerator utility producing an immutable 6-character alphanumeric code (e.g. AB7K9Q) excluding ambiguous characters (I, O, 0, 1) with collision retry checks.
- * OpenFeign Client Coordinator: Coordinates synchronous calls to flight-service (seat locking) and fare-service (price verification) before committing the local booking transaction.
- * Asynchronous Event Producer: On successful database commit, booking-service publishes BookingCreatedEvent to RabbitMQ topic exchange (booking.exchange) to trigger background notification workers.
- * Key Endpoints: POST /api/bookings (Create booking), GET /api/bookings/pnr/{pnr}, GET /api/bookings/{id}, DELETE /api/bookings/{id} (Cancellation & Seat release).

6. check-in-service | Airport Check-In & Boarding Pass Gate

Port: 8090 | Database: aerobook_checkin_db | Tables: check_ins, boarding_passes, qr_tokens

- * Architectural Responsibility: Manages the final traveler lifecycle phase: online 24-hour web check-in, seat assignment, digital boarding pass generation, and gate staff verification.
- * 24-Hour Check-In Rule: Validates via OpenFeign to booking-service that the PNR exists and that the flight departure is within 24 hours.
- * Digital QR Boarding Pass: Generates an encrypted, tamper-proof QR code payload containing PNR, passenger name, flight number, seat, gate, and SHA-256 digital signature.
- * Gate Staff Verification: Airport gate agents scan the boarding pass QR code at the departure gate; check-in-service validates the cryptographic signature and updates status from CHECKED_IN to BOARDED.
- * Key Endpoints: POST /api/check-ins (Initialize check-in), POST /api/check-ins/select-seat, GET /api/check-ins/{id}/boarding-pass, PUT /api/check-ins/{id}/board.

5. Inter-Service Communication & Event Messaging

Synchronous OpenFeign vs. Asynchronous RabbitMQ Event Bus

A critical design decision in microservice systems is choosing between synchronous request-reply and asynchronous event messaging. AeroBook employs a hybrid model that maximizes consistency where required and optimizes latency through decoupling.

SYNCHRONOUS PROTOCOL: Spring Cloud OpenFeign

- * When Used: For operations requiring immediate, real-time consistency where the calling service cannot proceed without an answer.
- * Key Feign Client Implementations in AeroBook:
 1. auth-service -> user-service (UserClient): Called during passenger registration to store demographic profiles.
 2. booking-service -> flight-service (FlightClient): Invokes /seats/lock to guarantee an inventory seat is reserved before taking payment.
 3. booking-service -> fare-service (FareClient): Invokes /fares/calculate to re-verify price quotes before charging credit cards.
 4. check-in-service -> booking-service (BookingClient): Verifies PNR validity and flight date within the 24-hour departure window.
- * Reliability Measures: Feign clients utilize Ribbon/Eureka client-side load balancing with 5-second connect timeouts and 10-second read timeouts.

ASYNCHRONOUS PROTOCOL: RabbitMQ AMQP Message Broker (Port 5672)

- * When Used: For background events that do not need to block the user's checkout HTTP response (eventual consistency).
- * Topic Exchange (booking.exchange): Produces structured JSON event envelopes with routing keys booking.created.* and booking.cancelled.*.
- * Consumer Workers Decoupled from Checkout:
 1. Email Notification Worker: Generates HTML booking confirmation receipts with flight itinerary and sends via SMTP.
 2. SMS Dispatcher Worker: Dispatches automated 6-character PNR text alerts to traveler mobile numbers.
 3. Analytics & Audit Sink: Ingests booking telemetry into auditing logs for airline revenue dashboards.
- * Dead-Letter Exchange (DLX): Failed messages are routed to a dead-letter queue (booking.dlq) after 3 retry attempts, preventing message loss.
- * Performance Benefit: Reduces ticket checkout response time from >2500ms (synchronous email/SMS) to under 120ms (instant AMQP handoff).

TRANSACTIONAL WALKTHROUGH: The 7-Step Booking Lifecycle

- Step 1: Passenger submits booking payload (flightNumber, travelDate, passengers, seatClass) to API Gateway (port 8083).
- Step 2: AuthenticationFilter validates JWT token and propagates X-User-Id header to booking-service (port 8088).
- Step 3: booking-service calls flight-service via OpenFeign to lock requested seats for 10 minutes (prevents double-booking).
- Step 4: booking-service calls fare-service via OpenFeign to verify dynamic ticket price (surge multiplier + cabin class).
- Step 5: booking-service calls Payment Gateway. If payment succeeds, booking transitions to CONFIRMED. If failed, seat lock is aborted.
- Step 6: booking-service generates unique 6-character PNR (AB7K9Q) and saves reservation in aerobook_booking_db.
- Step 7: booking-service publishes BookingCreatedEvent to RabbitMQ (port 5672) and returns HTTP 200 OK with PNR receipt to traveler.

6. Data Persistence & Cross-Cutting Architecture

Database-per-Service Isolation, Non-Null Serialization & Fault Tolerance

THE DATABASE-PER-SERVICE PATTERN: Why Separate Schemas?

A foundational rule of microservices is that services MUST NOT share database tables. In AeroBook, each service connects exclusively to its dedicated MySQL schema running on port 3306. This guarantees complete domain autonomy, zero cross-service table locks, and allows each team to evolve their schema independently.

aerobook_auth_db

Port: 3306 | Tables: credentials, roles, refresh_tokens

Stores hashed BCrypt credentials and JWT refresh token sessions. Strict index on email column.

aerobook_user_db

Port: 3306 | Tables: users, addresses, emergency_contacts

Contains traveler personal data, address, and passport expiration dates. Decoupled from credentials.

aerobook_flight_db

Port: 3306 | Tables: aircrafts, flights, seats, seat_locks

Maintains aircraft configurations, route schedules, and real-time seat lock state with 10-minute TTL.

aerobook_fare_db

Port: 3306 | Tables: fare_rules, base_fares, class_multipliers

Holds pricing coefficients, seasonal peak multipliers, and baggage fee rules.

aerobook_booking_db

Port: 3306 | Tables: bookings, passengers, payment_logs

Stores master reservations, traveler line items, payment audit tokens, and 6-character PNRs.

aerobook_checkin_db

Port: 3306 | Tables: check_ins, boarding_passes, qr_tokens

Contains check-in records, assigned seat codes, and digital boarding pass QR cryptographic hashes.

Cross-Cutting Concerns & Production Hardening

Platform-Wide Consistency Standards

Platform-Wide Jackson Property Inclusion (Non-Null Serialization)

All microservices configure `spring.jackson.default-property-inclusion=non_null` and use Swagger's `io.swagger.v3.core.util.Json.mapper()`. This eliminates empty fields like `"$ref": null` on HTTP Bearer security schemes, resolving Swagger UI resolver crashes platform-wide.

Global Exception Handling (@ControllerAdvice)

Every microservice encapsulates exceptions in a centralized `@RestControllerAdvice` handler. Any `ResourceNotFoundException`, `InvalidCredentialsException`, or `MethodArgumentNotValidException` is mapped into a uniform `ErrorResponse` DTO with timestamp, status code, error, and message.

OpenAPI 3 & Swagger UI Interactive Documentation

Every controller endpoint is documented with Swagger annotations (`@Operation`, `@ApiResponse`s, `@Parameter`, `@Tag`). The central API Gateway unifies all 7 microservice documentation endpoints into a single interactive portal at `http://localhost:8083/swagger-ui.html`.

Spring Boot Actuator Health Probes

Each service exposes liveness and readiness probes (`/actuator/health`). The API Gateway monitors downstream availability and Eureka deregisters unresponsive nodes automatically.

7. Microservices Topology Matrix & Architectural FAQ

Complete Technical Specification & Defense Review Guide

#	Service Name	Port	Engine / DB Schema	Outbound Feign Clients	Messaging
1	api-gateway	8083	Netty / Stateless (No DB)	Reverse Proxy Routing	None
2	auth-service	8082	Tomcat / aerobook_auth_db	-> user-service (UserClient)	None
3	user-service	8084	Tomcat / aerobook_user_db	None (Passive Vault)	None
4	flight-service	8087	Tomcat / aerobook_flight_db	None (Fleet Authority)	None
5	fare-service	8089	Tomcat / aerobook_fare_db	None (Pricing Engine)	None
6	booking-service	8088	Tomcat / aerobook_booking_db	-> flight-service, fare-service	AMQP Producer
7	check-in-service	8090	Tomcat / aerobook_checkin_db	-> booking-service (Verify PNR)	None

Architectural Defense & Technical Review FAQ

Key Answers for System Examiners & Evaluators

Q1: Why did you choose Spring Cloud Gateway WebFlux over Spring MVC Zuul/Tomcat?

Spring Cloud Gateway is built on non-blocking reactive Netty (Project Reactor). Zuul 1 and traditional Spring MVC use blocking thread-per-connection architectures that consume high memory under load. Netty allows the Gateway to handle 10x more concurrent connections on lower hardware resources.

Q2: How do you prevent distributed transaction failures without heavy 2PC/XA locks?

AeroBook implements the Saga orchestration pattern with compensating transactions. booking-service orchestrates seat locking in flight-service before charging payment. If payment fails, it immediately sends a compensating DELETE /seats/lock request to release inventory.

Q3: How do microservices authenticate requests if only the Gateway has the JWT filter?

The Gateway validates the JWT signature and extracts user claims. It then propagates trusted internal HTTP headers (X-User-Id, X-User-Role, X-User-Email) to downstream services. Downstream services read these headers without repeating expensive cryptographic verification on every hop.

